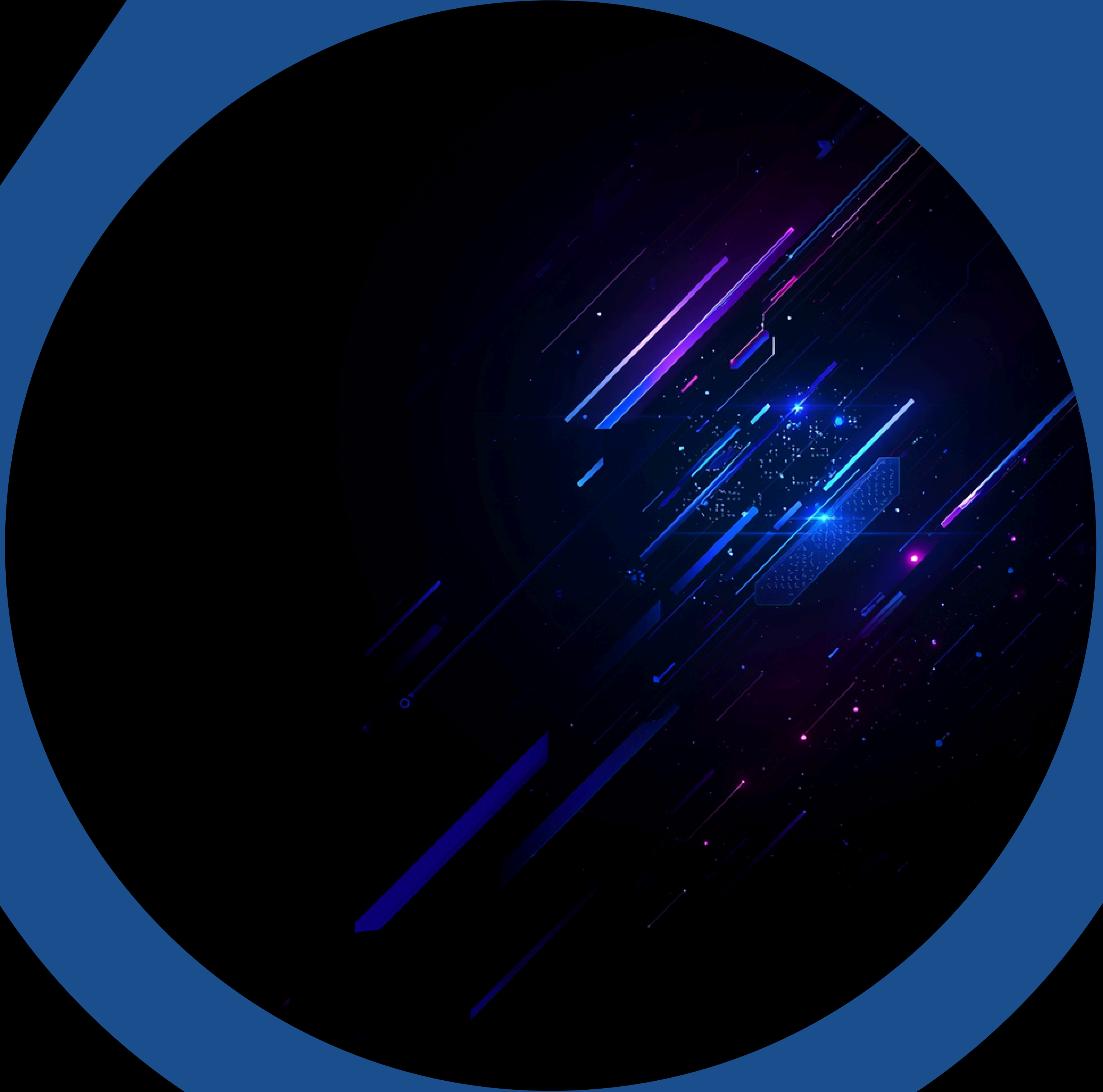
LILO: INNOVATING CODE EXECUTION

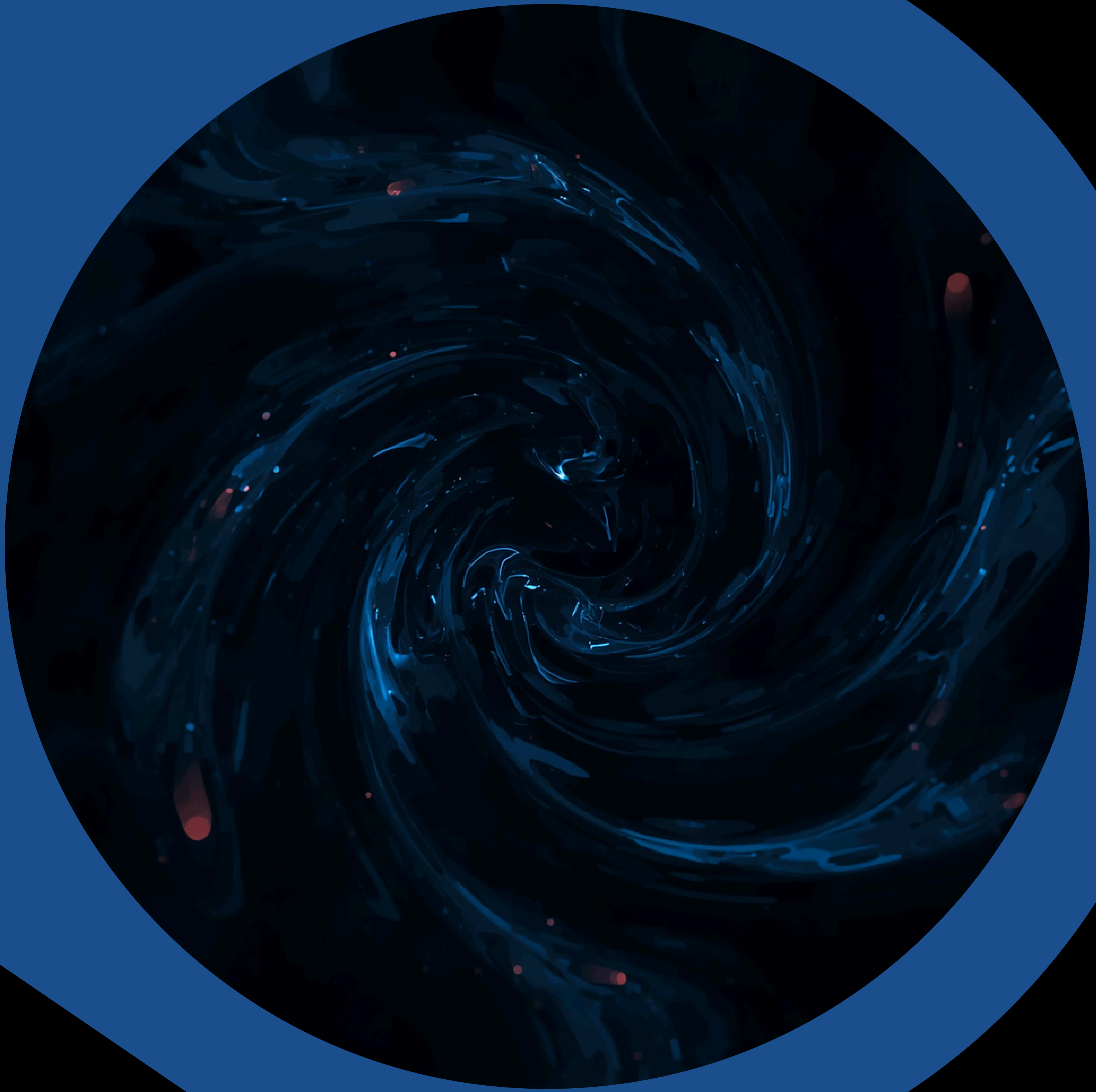
Killing the Cloud Sandbox



Max Myers, SRE Day Seattle, April 21, 2026

About Me

- SWE on Microsoft's Edge Web Platform Media team
- CEO & Co-Founder of LILO
- Building the code execution platform for early-career devs breaking into SWE.



Expectation vs. Reality

In the world of software development, **not every story** concludes with a heroic save. This post mortem reveals the truth behind our journey with the cloud platform. It’s a candid narrative about our challenges and mistakes, showcasing how we navigated through the complexities of shipping code without the anticipated success. Prepare to rethink your assumptions on project outcomes.

The Former Platform

Students work through lessons in the browser; our cloud sandbox runs their code against test cases.

Consequences

$60

First month's AWS cost

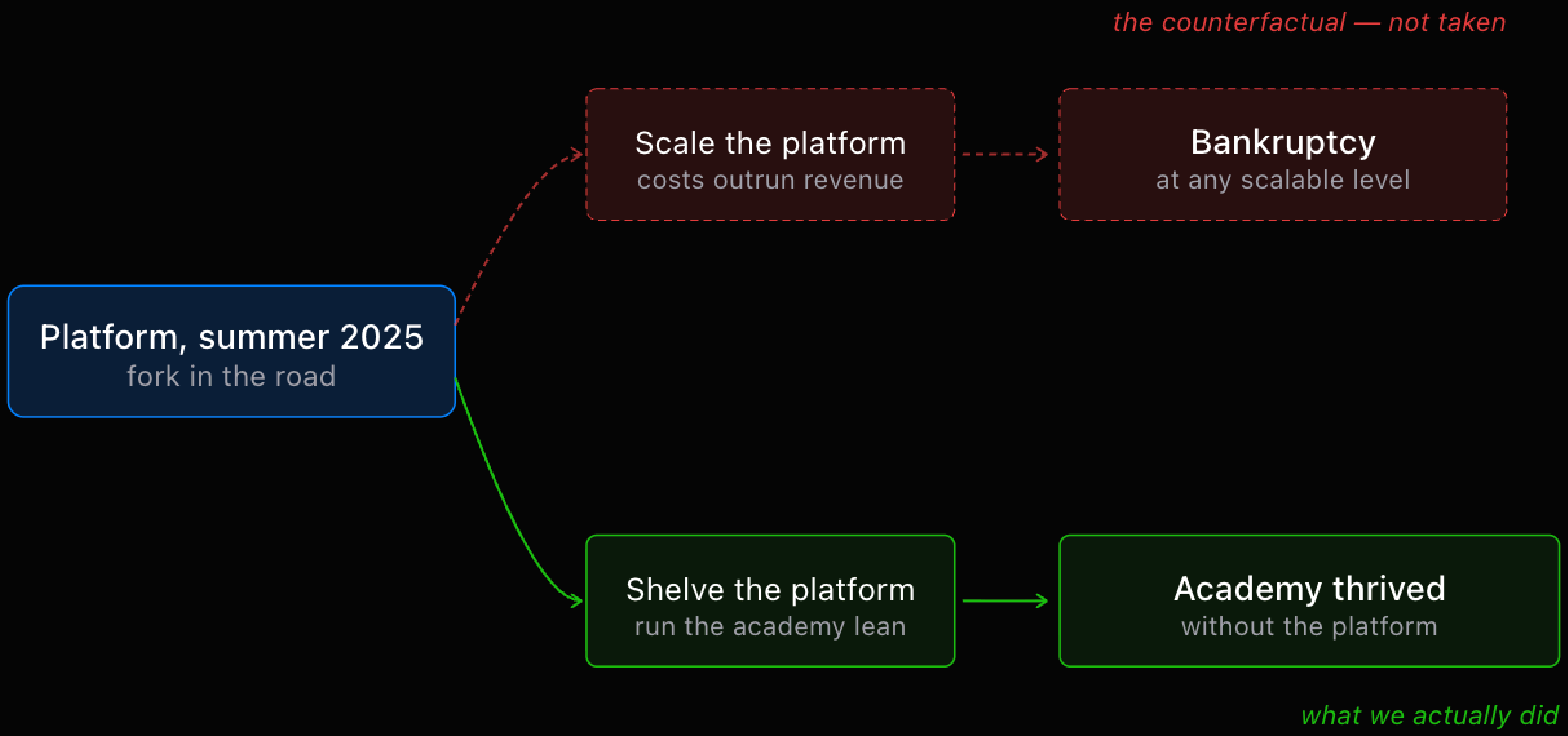
0.001 to 0.01

Industry standard cost per execution

1 to 3 seconds

Average submission latency

The Life and Death of the Platform



Restraint wasn't a failure — it was the right call.

Fork in the Road

Reduce execution cost
which path?

Option 1
Optimize infrastructure

Best case: −40% cost

- ✗ Cannot scale further
- ✗ No structural change
- ✗ Server bottleneck
- ✗ Diminishing returns

Option 2
Client-side WASM

Structural shift

- ✓ Scales with user growth
- ✓ Unlocks new architecture
- ✓ Lower marginal cost
- ✓ Improved performance

What is WASM?

Traditional JavaScript

JavaScript
Source code

→

JS engine
Parse, JIT compile

→

Browser
Runs the code

Scripting performance — hits a ceiling on compute-heavy work

With WebAssembly

C / C++ / Rust
Source code

→

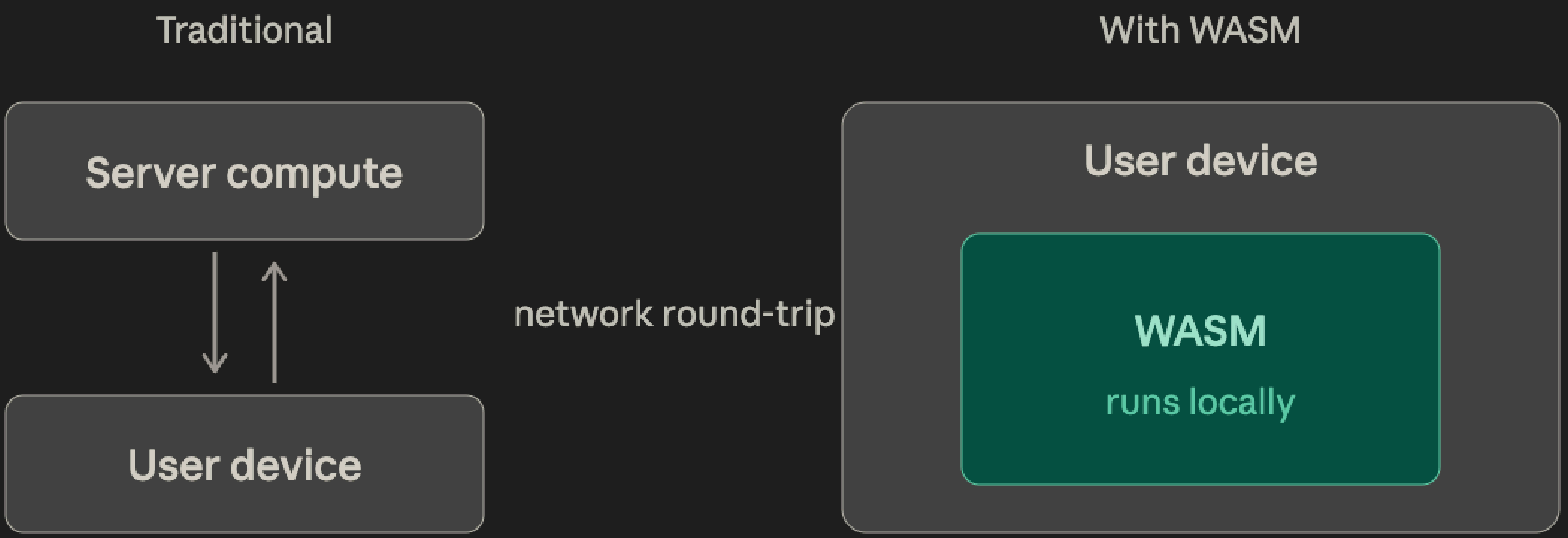
.wasm binary
Compiled bytecode

→

Browser
WASM runtime

Near-native performance — no scripting ceiling

Why WASM?



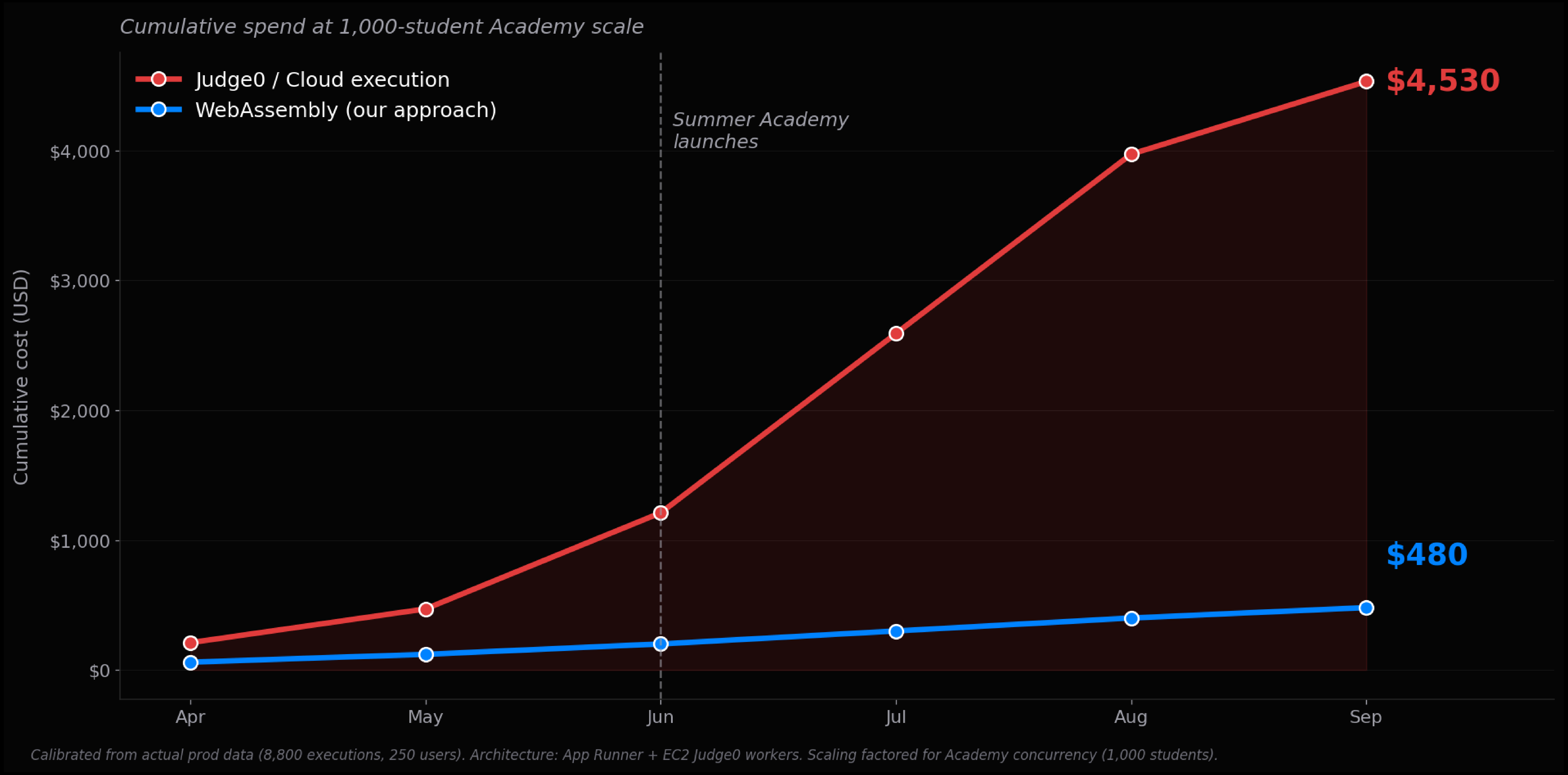
Cost efficiency

No server execution cost

Performance gains

Lower latency, no round-trip

Projected Costs



Java Challenges

1 The JVM isn't a flat runtime

Python

CPython
interpreter

Ships as one WASI component

Java / HotSpot JVM

- Dynamic classloader
- JIT compiler
- Threads, GC
- Host-bound stdlib

Virtual CPU + runtime services

2 CheerpJ is browser-only by design

CheerpJ

Browser APIs
DOM · Web Workers · IndexedDB · fetch()

Cloudflare Workers / WASI edge
no reachable path

The browser is the substrate. Strip it away, nothing to stand on.

Customer Personas

SELF-TAUGHT SAM

GRADED GRACE

COMPLIANCE CARL

$55 CHROMEBOOK, RURAL GHANA
LEARNS DSA ON HIS OWN THROUGH POWER OUTAGES

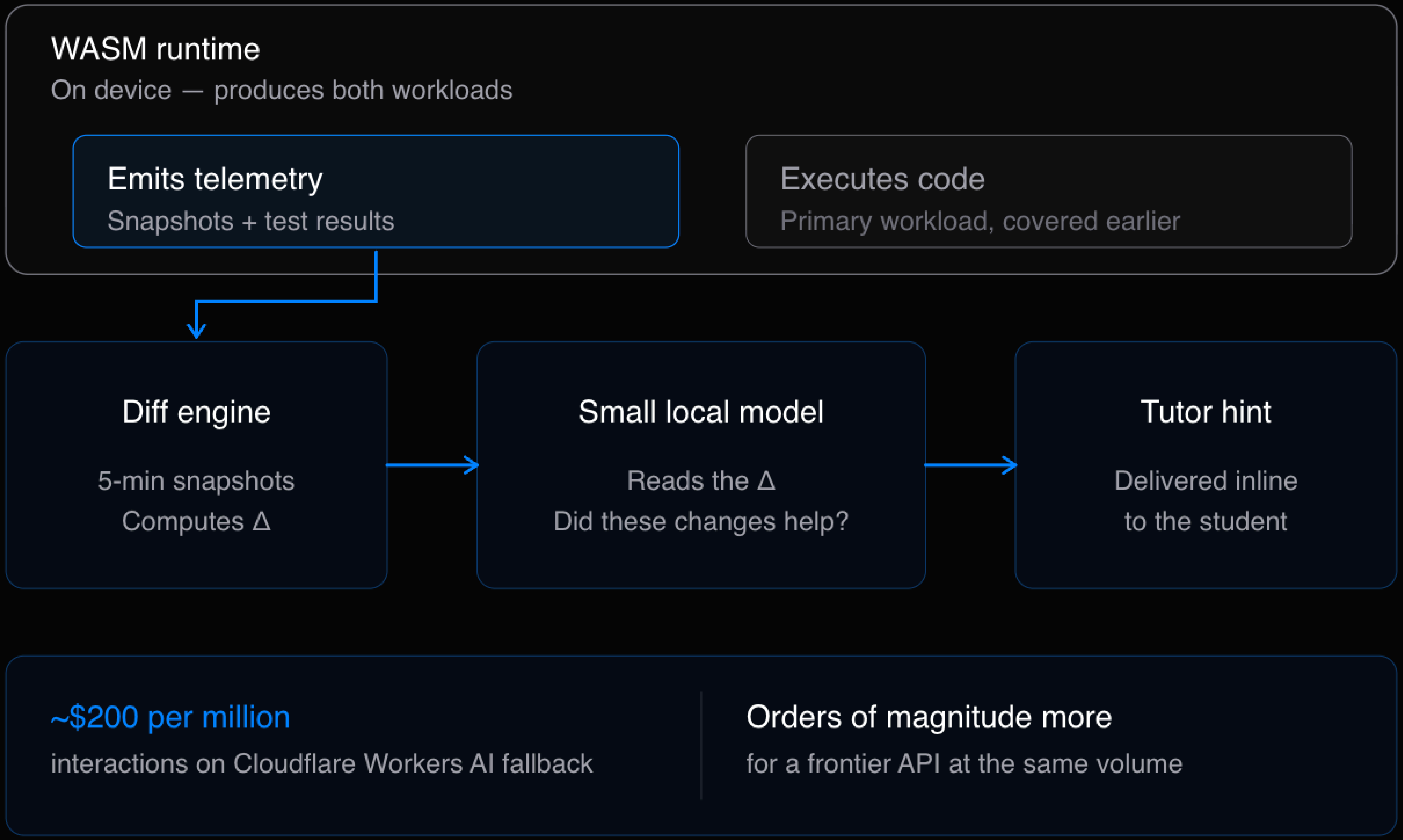
16GB MACBOOK, US UNIVERSITY CS
FULL STACK LOCAL EXECUTION + TUTOR
GRADED VIA CLOUD JUDGE

UNIVERSITY I.T.
COMPLIANCE + ANTI-CHEAT MANDATES
DRIVES THE CLOUD JUDGE

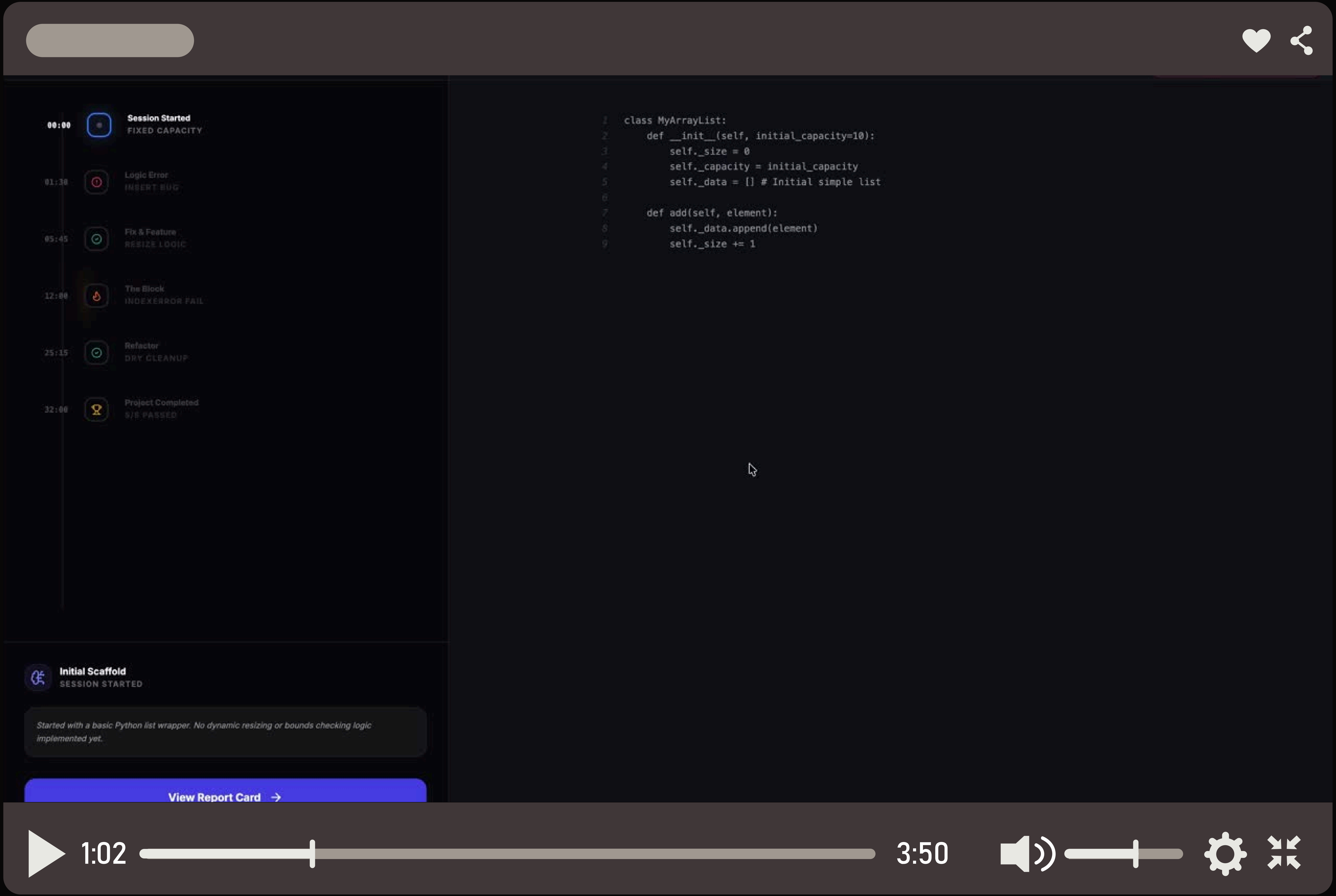
Self Taught Sam

$55 CHROMEBOOK, RURAL GHANA
LEARNS DSA ON HIS OWN THROUGH POWER OUTAGES

On-Device AI

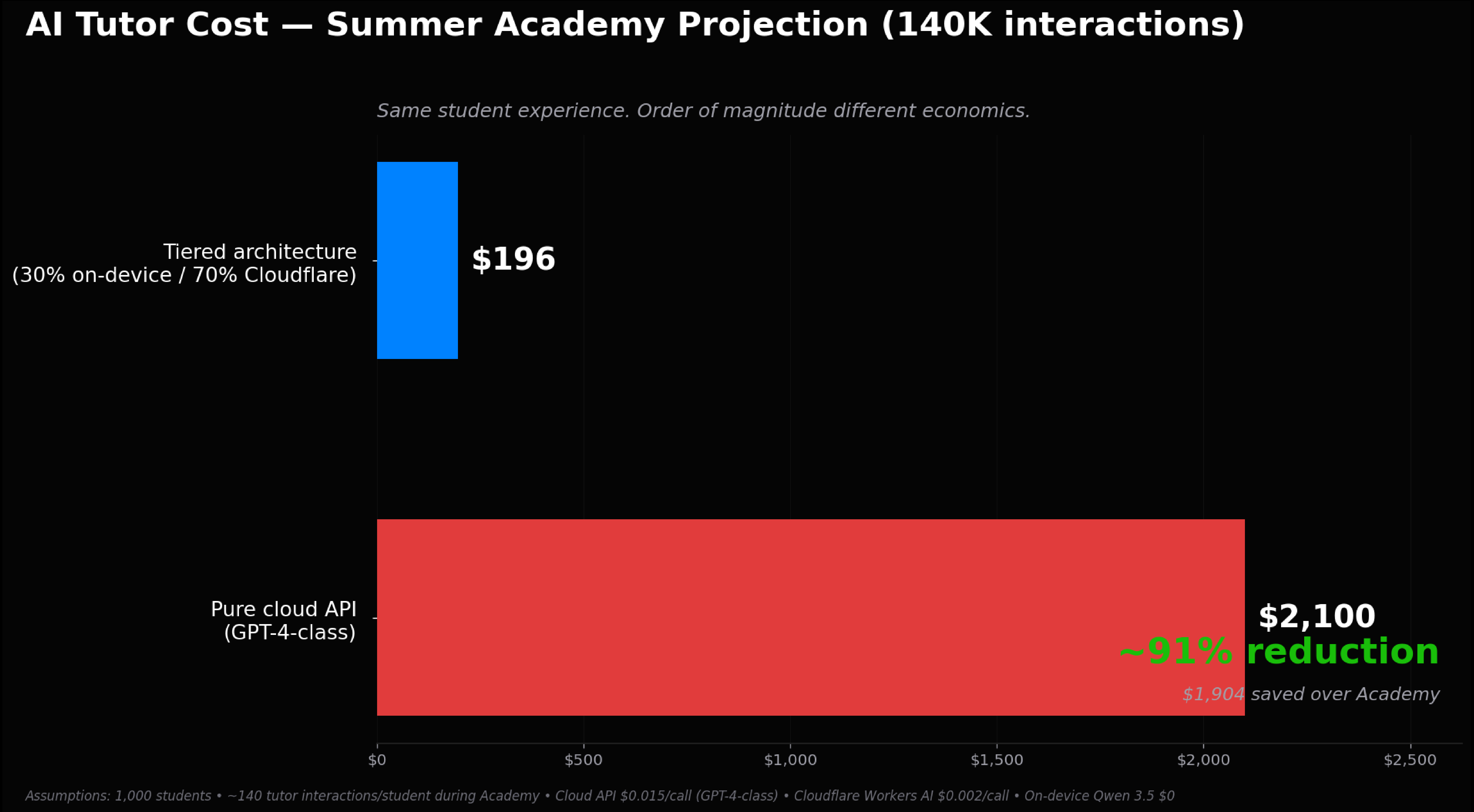


The AI isn't clever — it reads ground truth the WASM runtime already produced.



00:00
Session Started
FIXED CAPACITY
01:30
Logic Error
INSERT BUG
05:45
Fix & Feature
RESIZE LOGIC
12:00
The Block
INDEXERROR FAIL
25:15
Refactor
DRY CLEANUP
32:00
Project Completed
5/5 PASSED
class MyArrayList:
def __init__(self, initial_capacity=10):
self._size = 0
self._capacity = initial_capacity
self._data = [] # Initial simple list
def add(self, element):
self._data.append(element)
self._size += 1
Initial Scaffold
SESSION STARTED
Started with a basic Python list wrapper. No dynamic resizing or bounds checking logic implemented yet.
View Report Card →
1:02
3:50

On-Device AI's Effect



Looking Ahead

Integrating WASM didn't eliminate our challenges, it reshuffled them into two groups:
problems WASM couldn't absorb, and problems WASM introduced.

WASM Couldn’t Absorb...

Cloud service	Stays server-side for FERPA compliance with accredited universities.
Offline functionality	Carries an engineering tax, detailed in the on-device AI section.
Security concerns	Student devices can be tampered with, affecting grading accuracy.

The Cost Of Client-Side Execution

Payload limitations	The 10MB WASM ceiling restricts how large apps can be.
Device diversity	Heterogeneous devices force us to maintain two code paths.
Observability issues	When execution runs on-device, backend metrics go blind.

Thank You!